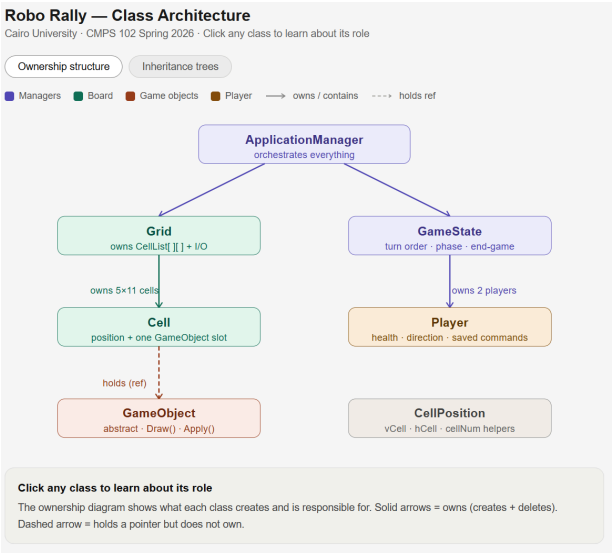
Cairo University
Faculty of Engineering
Computer Engineering Department
Programming Techniques
CMPS 102 Spring 2026

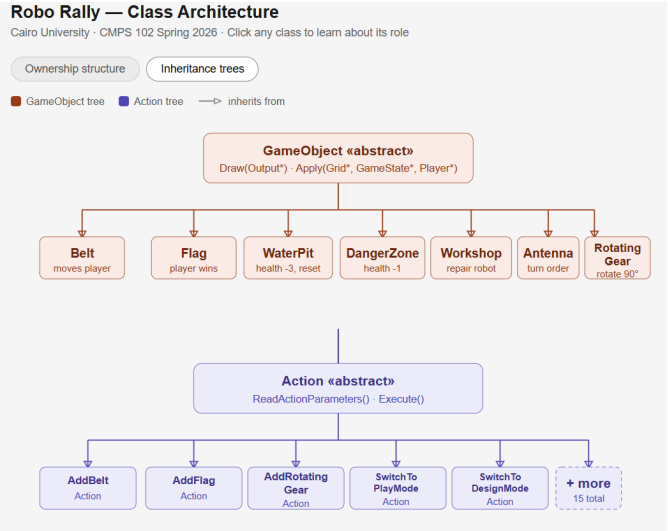
CMPS102 Programming Techniques Project

Robo Rally

Main Classes

You are given a ***code framework*** for **Phase 2** where we have ***partially*** written code of some of the project classes. You must **stick to** the **given design** (i.e. hierarchy of classes and the specified job of each class) and complete the given framework by either: extending some classes or inheriting from some classes (or even creating new base classes)





Open this link for interactive illustration for the class architecture:
https://khalidmamdou7.github.io/pt-roborally-project-diagram/

Class Responsibilities

The class design is fixed. You may add helper functions inside a class, but you may not move responsibilities from one class to another. The rules below are enforced during grading.

Input and Output

- Input and Output are the **only** classes that may access the CMU graphics library.
- All user input (mouse clicks, keyboard entry) must pass through Input member functions.
- All drawing and message display must pass through Output member functions.
- No console input or output anywhere in the project.

ApplicationManager

- Creates and owns the Grid and GameState objects.
- Each iteration of the main loop: reads the user action, creates the matching Action object, calls Execute(), then calls UpdateInterface().
- Does not contain game logic or drawing code. It only creates Action objects and delegates.

Action and subclasses

- One class per toolbar operation. Each class overrides ReadActionParameters() and Execute().
- ReadActionParameters() calls Input functions to read what the user has selected.
- Execute() calls ReadActionParameters() first, then performs the operation using Grid, GameState, and GameObject functions.
- Actions do not draw anything directly — they delegate to Output through Grid or Player.

Grid

- Owns the 2D CellList array. No other class may access CellList directly.
- Provides public functions (AddObjectToCell, RemoveObjectFromCell, UpdatePlayerCell, etc.) that give controlled access to the board.
- Functions that must loop over CellList (e.g. SaveAll) may do so, but they only call virtual functions on each cell or game object — they do not contain game logic themselves.
- Does not own Player objects. Players are owned by GameState.

GameState

- Owns all Player objects.
- Tracks whose turn it is, the current phase, and whether the game has ended.
- Does not own or access CellList. Board operations go through Grid.

GameObject and subclasses

- Each subclass overrides Draw() to delegate to the appropriate Output function.
- Each subclass overrides Apply(Grid*, GameState*, Player*) to implement its effect on a player. Logic lives inside Apply() — not in the calling code.
- Each subclass overrides Save() and Read() for file I/O.
- Never use dynamic_cast or a type enum as a substitute for virtual functions.

Cell

- Holds a pointer to at most one GameObject.
- Provides Has*() helpers (HasBelt, HasFlag, HasWaterPit, HasDangerZone) using dynamic_cast — this is the one sanctioned use of dynamic_cast in the project.
- DrawCellOrWaterPitOrDangerZone() and DrawGameObject() call the virtual Draw() of whichever object the cell contains.

Player

- Stores position (via a Cell pointer), direction, health, and saved commands.
- Draw() and ClearDrawing() delegate to Output.
- Move() executes the player's saved commands one by one, then triggers the Apply() of any game object at the final cell.

Game Rules Reference

Grid

- 55 cells arranged in 5 rows and 11 columns.
- Cells are numbered 1–55, from bottom-left to top-right (left-to-right within each row, rows ordered bottom-up).
- Cell 1 is the start cell (bottom-left). Game objects cannot be placed in cell 1 or cell 55.

Player

- The game has two players. Both start at cell 1 (the start cell).
- Each player has a direction (initially RIGHT) and health (initially 10).
- The number of commands a player may select per round is Min(5, health).
- Players take turns; turn order is determined by the Antenna at the start of each round.

Movement commands

Commands are randomly generated each round. The pool size equals the player's current health. Available commands are:

- **Move Forward X steps** ($X \in \{1, 2, 3\}$): move X cells in the current facing direction.
- **Move Backward X steps** ($X \in \{1, 2, 3\}$): move X cells opposite to the facing direction.
- **Rotate Right:** rotate 90° clockwise.
- **Rotate Left:** rotate 90° counter-clockwise.

After selecting commands (up to Min(5, health)), the player executes them in order. After each individual command, the game object at the robot's new cell (if any) is applied — **except** for Workshop, which is applied only after all commands in the round have been executed.

Example Scenarios

Example Scenario 1: AddBeltAction (in Design Mode)

Here is an example scenario for **adding a belt** in the Grid in Design Mode. It is performed through the four steps mentioned in '**Appendix A - implementation guidelines**' section. These four steps are in the "main" function of phase 2 code. You **must NOT** change the "main" function of **phase 2**.
The 4 steps are as follows:

Step 1: Get user input

1- The **ApplicationManager** calls the **Input** class and waits for user action.
2- The user clicks on the **"Add Belt" icon** in the toolbar to add a belt.
3- The **Input** class checks the area of the click and recognizes that it is a "add belt" operation. It returns `ADD_BELT` (an **"enum"** value representing the **ActionType**) to the manager.

Step 2: Create a suitable action

1- **ApplicationManager::ExecuteAction(ActionType)** is called to create an action object of type **AddBeltAction** class:

```
Action* pAct = NULL;
// in a switch case on the passed ActionType
case ADD_BELT:      pAct = new AddBeltAction(this);
```

Step 3: Execute the action

1- **ApplicationManager::ExecuteAction(…)** calls the virtual **AddBeltAction::Execute( )**

```
pAct->Execute(); // Execute the AddBeltAction object
```

2- **AddBeltAction::Execute( )**

a. Calls **AddBeltAction::ReadActionParameters( )** which calls function **GetCellClicked()** twice from the **Input** class to get the action parameters of **AddBeltAction** (i.e. the start and end cell positions, `startPos` and `endPos`).

Notice that when **AddBeltAction** wants to print messages to the user on the status bar, it calls some functions from the **Output** class.

b. Creates (allocates) an object of class **Belt**:

```
Belt * pBelt= new Belt(startPos, endPos);
```

c. Asks the **Grid** to set the created belt object to the GameObject pointer of its Cell (the start cell of the belt) by calling **Grid::AddObjectToCell (…)** function and passing the new belt to it:

```
pGrid->AddObjectToCell(pBelt);
```

Function Grid:: AddObjectToCell (`GameObject * pNewObject`) will do the following (Note: the passed pNewObject now points to the newly created belt object):

1. Get the cell position of the belt object (which represents its **start** cell):

```
CellPosition pos = pNewObject->GetPosition();
```

2. **Set the passed belt object to the GameObject pointer of its start cell in the CellList:**

```
CellList[pos.VCell()][pos.HCell()]->SetGameObject(pNewObject);
```

Step 4: Update Interface

1- **ApplicationManager::UpdateInterface( )** calls **Grid:: UpdateInterface ( )**

2- The **UpdateInterface()** of the **Grid** class *in the Design Mode* redraws all cells with their game objects (flags, water pits, danger zones, belts, workshops, antennas and rotating gears) using the following steps:

   a. **It iterates on each Cell in the CellList of Grid class and Call the following function to redraw each cell and either its water pit or danger zone if it contains one:**

```
CellList[i][j]->DrawCellOrWaterPitOrDangerZone(pOut);
```

   b. **It iterates again on each Cell in the CellList and Call the following function to redraw the game object (excluding water pits and danger zones) of each cell:**

```
CellList[i][j]->DrawGameObject(pOut);
```

- **Notes:**

   a. **UpdateInterface()** draws all cells with water pits and danger zones first before the other game objects to avoid drawing a cell above a drawn object.

   b. **DrawCellOrWaterPitOrDangerZone(…)** function of class **Cell** calls the **Draw(…) virtual function** of the **GameObject** class if the cell contains a water-pit/danger-zone game object. Similarly, the **DrawGameObject(…)** function of class **Cell** calls the **Draw(…) virtual function** of class **GameObject** if the cell contains any of the other game objects.

   c. The **Draw(…)** function of **GameObject** class is a **pure virtual function** which is overridden in class **Belt** to call **Output::DrawBelt(…)** of class Output that is responsible for Drawing the belts in the Interface.

Example Scenario 2: Save Grid Action (in Create-Grid Mode)

- **In General, Save/Open** has NO relation to the **Input** or **Output** classes. They save/open grids to/from **files** not the graphical window.

- Here we explain the calling sequence in the execution of '**SaveGridAction'** action as an example. **Note** the **responsibility of each class** and how each class does only its job or responsibility.

- There is a **save function** in **Grid** class and in each type of **GameObject** but each function performs a different job:

1. **GameObject :: Save (… , Type)**
   It is a **virtual** function in **GameObject** class. Each class derived from GameObject class should **override** it with its own implementation to save itself because each GameObject type has different information and hence a different way or logic to save itself. The function takes a "*Type" parameter* which could be an *enum* or an *integer* that represents the GameObject type that should be saved (either flag, water pit, danger zone, belt, workshop, antenna or rotating gear). So, each GameObject class type will check the "Type" parameter sent to the function. If it is the same type of the class, it will save its information to the file, otherwise, return without saving.

   This "**Type**" parameter is sent because as shown in the "**File Format**" section mentioned below, all **flags** should be saved first, then all **water pits**, then all **danger zones**, then all **belts**, then all **workshops**, then all **antennas**, then all **rotating gears**.

2. **Grid :: SaveAll (… , Type)**
   It is the function responsible for **calling** the **GameObject :: Save(…, Type)** function for the GameObject of each cell (if any) in the Grid's **CellList** because **Grid class** is the only class that has **CellList** and no one else can access it. Note that it only gets the GameObject of each cell and calls function save of the game object; **ONLY calling without making the save logic itself** (not the responsibility of **Grid** but the responsibility of each game object class). **This note is important and has a huge grade percentage**.

   Note: dynamic_cast is NOT needed in this function because polymorphism will automatically call the Save function of the correct object type for each GameObject pointer in the list.

3. **SaveGridAction :: Execute( )**
   It does the following:
   - ❑first reads action parameters (i.e. the filename)
   - ❑then opens the file
   - ❑and calls **Grid::SaveAll (…, FlagsType)** to save all flags
   - ❑and calls **Grid::SaveAll (…, WaterPitsType)** to same all water pits
   - ❑and calls **Grid::SaveAll (…, DangerZonesType)** to same all danger zones
   - ❑and calls **Grid::SaveAll (…, BeltsType)** to same all belts
   - ❑and calls **Grid::SaveAll (…, WorkshopsType)** to same all workshops
   - ❑and calls **Grid::SaveAll (…, AntennasType)** to same all antennas
   - ❑and calls **Grid::SaveAll (…, RotatingGears)** to same all rotating gears
   - ❑then closes the file

   Note: if any information is available for SaveGridAction class without breaking class responsibilities, it should write it to the file by itself.
   Important Note: Don't abuse the "*Type*" enum that you will create for this function. Whenever virtual functions and polymorphism could be applied, apply them. If any use of Type enum in your project will replace virtual functions and break class responsibilities, this will be grade-penalized.

File Format

The "Grid" File Format:

Your application should be able to **save/open a grid** to/from a simple text file. In this section, the file format is described together with an example.

- **File Format**

```
Number_of_Flags(n1)
Flag_Cell
Number_of_WaterPits(n2)
WaterPit_1_Cell
WaterPit_2_Cell
……..
WaterPit_n2_Cell
Number_of_DangerZones(n3)
DangerZone_1_Cell
DangerZone_2_Cell
……..
DangerZone_n3_Cell
Number_of_Belts(n4)
Belt_1_start_cell   Belt_1_end_cell
Belt_2_start_cell   Belt_2_end_cell
……..
Belt_n4_start_cell  Belt_n4_end_cell
Number_of_Workshops(n5)
Workshop_1_Cell
Workshop_2_Cell
……..
Workshop_n5_Cell
Number_of_Antennas(n6)
Antenna_Cell
Number_of_RotatingGears(n7)
RotatingGear_1_cell        RotatingGear_1_rotation_direction
RotatingGear_2_cell        RotatingGear_2_rotation_direction
……..
RotatingGear_n7_cell       RotatingGear_n7_rotation_direction
```

- **Example:** The grid file looks like that (*comments in green are just for explaining the example*)

```
1//number of flags (should always be 1)
54// cell (54)
2  //number of water pits
10// cell (10)
53// cell (53)
2  //number of danger zones
20// cell (20)
29// cell (29)
2//number of belts
7    29// start cell (7), end cell (29)
14  36// start cell (14), end cell (36)
1//number of workshops
49// cell (33)
1//number of antennas (should always be 1)
49// cell (49)
3 //number of rotating gears
5 1// rotating gear in cell 5. Rotation direction is clockwise.
23 0// rotating gear in cell 23. Rotation direction is counterclockwise.
44 1// rotating gear in cell 5. Rotation direction is clockwise.
```

Notes:

- You are allowed to modify this file format if necessary **but after instructor approval**.
- **The "Open" Action**:

  For lines in the above file, For Example for the belts: the **OpenAction** first **creates** (allocates) an empty object of that type of belt. Then, it **calls** **GameObject::Read** virtual function that can be overridden in the class of each game object type to make the object load its data from the opened file by itself (its job). Then, it **calls** **Grid::AddObjectToCell** to add the created belt object to a cell in the **CellList**.

Project Phase2

Partially-implemented code frameworks for Phase 2 are given to you to complete them.

For **fast navigation** in the given code in **Visual Studio**, you may need the following:

- **F12 (go to definition):** to go to definition of functions (code body), variables, …etc.
- **"Ctrl" then "Minus":** to return to the previous location of the cursor.

Phase 2 (Project Delivery)

In this phase, the completed **I/O classes**, **CellPosition class**, **DEFS.h** and **UI_Info.h** (**without phase 1 test code**) should be added to the project **framework code** given for phase 2 (and the **images folder** should be copied too) and the remaining classes of phase 2 should be implemented. Start by implementing the base classes then move to derived classes.

In the given code of phase 2, AddBeltAction of Design mode is almost fully implemented (needs only validation). In addition, some base classes are partially implemented with some guiding comments to follow.

You are required to:

- **Complete the Implementation of functions marked with ///TODO comment** as it is described in the comments. These are some useful functions that you can use in implementation of phase 2.
- **Complete all the classes mentioned in the "Main Classes Section"** with full implementation of their functions and finalize the project to perform all the operations mentioned in the project documents in the 2 modes.
- **You may need to add more classes to make the code more organized and object-oriented** but you are NOT allowed to change the initial classes' design, hierarchy and responsibilities we mentioned in the "**Main Classes Section**".

Phase 2 Deliverables:

(1) **Workload division**: a **printed page** containing team information and a table that contains members' names and the parts each member has implemented.

(2) Online Submission: a zip file containing the following:

a. **ID.txt file**. (Information about the team: names, IDs, team email)
b. **Workload division** (described in the previous point)
c. **The project phase 2 code and resources files** (images, saved files, …etc.).
d. **Sample grid files:** at least **three** different grids. For each grid, provide:
   i. Grid text file (created by save operation)
   ii. Grid screenshot for the grid generated by your program

Note that The online submission time will be the same for all project teams. **No modifications are allowed after the online submission.** After that, a face-to-face **discussion** of the project will be held.

Phase 2 Evaluation Criteria

Percentages are added according to the **difficulty** of the task, so to divide the project load **equally** on team members, each member should take actions that their percentages add up to about **25%** of phase 2 (if 4 students in the team).

Toolbar Operations [45%]

- Each operation percentage is mentioned below.
- **[39%]** Design Mode
  - **[8%]** AddBelt
  - **[3%]** Add Water Pit & Danger zone & Workshop & Rotating Gear
  - **[2%]** Add Flag
  - **[1%]** Add Antenna
  - **[8%]** Copy & Cut and Paste Game object
  - **[3%]** Delete Game Object
  - **[6%]** Save
  - **[6%]** Load
  - **[2%]** Switch to play mode
- **[1%]** Exit in Play Mode and Exit in Design Mode(Exit icon in each mode)
- **[5%]** Play Mode
  - **[1%]** Switch back to design mode (this icon in PlayMode)
  - **[3%]** New game
  - **[1%]** Display random commands

Game Logic [50%]

- Any percentage mentioned next to a Game Object means: handing all the Logic of this object in the game:
  - **[4%]** Display Players information
  - **[3%]** Player Movement (Select commands)
  - **[5%]** Player Movement (Execute commands)
  - **[5%]** Reboot and repair
  - **[3%]** Execute Flag Action
  - **[8%]** Execute Antenna Action
  - **[5%]** Execute Belt Action
  - **[3%]** Execute Water pit Action
  - **[3%]** Execute Danger zone Action
  - **[5%]** Rotating Gear
  - **[6%]** Workshop (for repair)

Code Organization & Style [5%]

- ❑ Every class in .h and .cpp files
- ❑ Variable naming
- ❑ Indentation & Descriptive Comments
- ❑ Note: this percentage is divided between team members (Each should make the code organization of his code part)

Bonus [10% maximum]

Workshop not just for repairing

In this theme, a player's robot can hold equipment and consumables. Robots can visit workshops on the map to repair themselves and **purchase additional equipment or consumables (one-time use)**.

- **Devices** :
  - Extended Memory **[3%]**: Allows a robot to perform 6 commands instead of 5.
- **Consumables** (one-time use appears in the action menu before assigning the commands):
  - Toolkit **[3%]** : Repairs the robot without visiting a workshop.
  - Hack Device **[4%]**: Prevents the opponent from playing during the current round.
- **Weapons (relates to shooting phase - check it first) [2%]** :
  - Double Laser: Can be purchased at workshops to deal 2 damage points.

Shooting phase [7%] :

- After movement commands are executed, robots enter a Shooting Phase.
- Each robot starts with one piece of equipment, a basic laser.
- If a robot directly faces its opponent in the same row or column, it fires its laser.
- Shooting mechanics are as follows:
  - Basic Laser: Deals 1 point of damage to the opponent.
  - Double Laser (optional, if purchased in a workshop): Deals 2 points of damage if equipped.
  - - If a robot successfully hits its opponent, a message should appear "You hit another player, click to continue is displayed."

General Evaluation Criteria for any Operation:

1. **Compilation Errors → MINUS 50%** of Operation Grade
   - ❑ The remaining 50% will be on logic and object-oriented concepts (see **point no. 3**)
2. **Not Running** (runtime error in its **basic functionality**) → **MINUS 40%** of Operation Grade
   - ❑ The remaining 60% will be on logic and object-oriented concepts (see **point no. 3**)
   - ❑ If we found runtime errors but in corner (not basic) cases, that will be part of the grade but not the whole 40%.
3. **Missing Object-Oriented Concepts → MINUS 30%** of Operation Grade
   - ❑ **Separate class** for each item and action
   - ❑ Each class does its **job**. No class is performing the job of another class.
   - ❑ **Polymorphism:** use of pointers and virtual functions
   - ❑ **See the "Implementation Guidelines" in the Appendix which contains all the common mistakes that violate object-oriented concepts.**
4. **For each corner case** that is not working → **MINUS 10% to 20%** of the Operation Grade according to instruction evaluation.

Note: The code of any operation does NOT compensate for the absence of any other operation.

Individuals Evaluation:

Each member must be responsible for writing some project modules (e.g. some classes or some functions) and must answer some questions showing that he/she understands both the program logic and the implementation details. The work load between team members must be almost equal.

✔ The grade of **each student** will be divided as follows:

- **[70%]** of the student grade is on his individual work (the project part he was responsible for).
- **[25%]** of the student grade is on integrating his work with the work of ALL students who Finished or nearly finished their project part.
- **[5%]** of the student grade is in cooperation with other team members and helping them with their problems in their code parts (helping does NOT mean implementing their parts).

✔ You should **inform the TAs** before the deadline **with sufficient time (some weeks before)** if any problems occurred between members to be able to give warnings and take actions.

✔ If one or more members didn't make their work, the other members will NOT be affected **as long as:** i) the students informed the TAs about this problem with a sufficient time before the deadline (to give warnings and take actions), ii) the students who finished their part integrated all their parts together AND their part can be tested easily to see the output.

✔ If a student couldn't finish his part, try to still integrate it to at least take the integration grade.

APPENDIX A

[I] Implementation Guidelines

- **Any user operation is performed in 4 steps:**
  - Get user action type.
  - Create a suitable action object for that action type.
  - Execute the action (i.e. function **Action::Execute()** which first calls **ReadActionParameters()** then executes the action).
  - Update Interface which updates the drawings on the window after each executed action

- **Use of Pointers/References**: Nearly all the parameters passed/returned to/from the functions should be pointers/references to be able to exploit **polymorphism** and **virtual** functions. **GameObject** in a cell should be a **Base Class Pointers (GameObject)** to be able to point to any game object type. Many class members should be pointers for the same reason.

- **Classes' responsibilities:** Each class must perform tasks that are related to its responsibilities only. No class performs the tasks of another class. For example, when class **Belt** needs to draw itself on the GUI, it calls function ***Output::DrawBelt*** because dealing with the GUI window is the responsibly of class **Output**. Similarly, class **Grid** must not contain any logic. It only should call functions. Read the "**main classes**" section to know the responsibility of each class.

- **Abusing Getters:** Don't use getters to get data members of a class to make its job inside another class. This breaks the classes' responsibilities rule. For example, do NOT add in **Grid** class function **CellList()** that gets the 2D array of cells to the other classes to loop on it and use it there. **CellList** and looping on it are the responsibility of the Grid class only.

- **Virtual Functions:** In general, when you find some functionality (e.g. saving) that has different implementation based on each type, you should make it a virtual function in the base class and override it in each derived class with its own implementation.
  - A common mistake here is the **abuse** of **dynamic_cast** (or similar implementations like **type** data member) by checking the object type outside the class and perform this class's job there (this job should be inside that class in a virtual member function in it).
  - This does not mean you should never use **dynamic_cast** but do NOT use it in a way that breaks the constraint of class responsibilities or replaces the use of virtual functions.

- **Not all the actions** need to add a corresponding function inside **Grid**. This will make **Grid** perform the responsibility of these actions. However, some actions need to loop on **CellList** (e.g. ***SaveGridAction*** …etc.). In this case only (looping on Game Objects), you can add functions for them in **Grid** that loop on the list and just call functions **without making any further logic**.

- You are not allowed to use **global variables** in your implemented part of the project, use passing variables as function parameters instead. That is better from a software engineering point of view.

- You need to get instructor approval before using **friendships.**

[II] Workload Division Guidelines

Workload must be distributed among team members. The first question to the team at the project discussion and evaluation is "who is responsible for what?" An answer like" we all worked together" is a **failure** and will be penalized.

Here is a recommended way to divide the work based on **Actions**

- Divide workload by assigning some actions to each team member. Each member takes an action, should make any needed changes in any class involved in that action then run and try this action and see if it performs its operation correctly then move to another action.

- For example, the member who takes action **'SaveGrid'** should create **'SaveGridAction'** and write the code related to **SaveGridAction** inside **'Grid'** and **'GameObject'** derived classes. Then run and check if the game objects are successfully saved. Don't wait for the whole project to finish to run and test your implemented action.

- It is recommended to give similar actions to the same member because they have similar implementations.

- After finishing and trying a few related actions, it's recommended to integrate them with the last integrated version and any subsequent divided action should increment on this project version and so on. We call this ***'Incremental Implementation'***.

- It's recommended to first divide the actions that other actions depend on (e.g. adding and creating grids) then integrate before dividing the rest of the actions.

- In the game logic, you may divide by object type, …etc. Each member takes an object type and totally handles it.